CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy

Nathan Dowlin[1] NDOWLIN@PRINCETON.EDU
Department of Mathematics, Princeton University

Ran Gilad-Bachrach RANG@MICROSOFT.COM
Kim Laine KIM.LAINE@MICROSOFT.COM
Kristin Lauter KLAUTER@MICROSOFT.COM
Michael Naehrig MNAEHRIG@MICROSOFT.COM
John Wernsing JOHN.WERNSING@MICROSOFT.COM
Microsoft Research, Redmond

Abstract

Applying machine learning to a problem which involves medical, financial, or other types of sensitive data, not only requires accurate predictions but also careful attention to maintaining data privacy and security. Legal and ethical requirements may prevent the use of cloud-based machine learning solutions for such tasks. In this work, we will present a method to convert learned neural networks to *CryptoNets*, neural networks that can be applied to encrypted data. This allows a data owner to send their data in an encrypted form to a cloud service that hosts the network. The encryption ensures that the data remains confidential since the cloud does not have access to the keys needed to decrypt it. Nevertheless, we will show that the cloud service is capable of applying the neural network to the encrypted data to make encrypted predictions, and also return them in encrypted form. These encrypted predictions can be sent back to the owner of the secret key who can decrypt them. Therefore, the cloud service does not gain any information about the raw data nor about the prediction it made. We demonstrate *CryptoNets* on the MNIST optical character recognition tasks. CryptoNets achieve 99% accuracy and can make around 59000 predictions per hour on a single PC. Therefore, they allow high throughput, accurate, and private predictions.

Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 2016. JMLR: W&CP volume 48. Copyright 2016 by the author(s).

1. Introduction

Consider a hospital that would like to use a cloud service to predict the probability of readmission of a patient within the next 30 days, in order to improve the quality of care and to reduce costs. Due to ethical and legal requirements regarding the confidentiality of patient information, the hospital might be prohibited from using such a service. In this work we present a way by which the hospital can use this valuable service without sacrificing patient privacy. In the proposed protocol, the hospital encrypts the private information and sends it in encrypted form to the prediction provider, referred to as *the cloud* in our discussion below. The cloud is able to compute the prediction over the encrypted data records and sends back the results that the hospital can decrypt and read. The encryption scheme uses a public key for encryption and a secret key (private key) for decryption. It is important to note that the cloud does not have access to the secret key, so it cannot decrypt the data nor can it decrypt the prediction. The only information it obtains during the process is that it did perform a prediction on behalf of the hospital. Hence, the cloud can charge the hospital for its services, but does not learn anything about the patient's medical files or the predicted outcomes. This procedure allows for private and secure predictions without requiring the establishment of trust between the data owner and the service provider. This may have applications in fields such as health, finance, business, and possibly others.

It is important to note that this work focuses on the inference stage. We make the assumption that the cloud already has a model. In our case it would be a neural network that

[1]This work was done while the first author was at Microsoft Research, Redmond

CryptoNets：将神经网络应用于加密数据，实现高吞吐量和准确性

Nathan Dowlin[1] NDOWLIN@PRINCETON.EDU
普林斯顿大学数学系

Ran Gilad-Bachrach RANG@MICROSOFT.COM
金·莱恩 KIM.LAINE@MICROSOFT.COM
克莉斯汀·劳特 KLAUTER@MICROSOFT.COM
迈克尔·纳赫里格 MNAEHRIG@MICROSOFT.COM
约翰·韦恩斯林 JOHN.WERNSING@MICROSOFT.COM
微软研究院，雷德蒙德

摘要

将机器学习应用于涉及医疗、金融或其他类型敏感数据的问题，不仅需要准确的预测，还需要仔细关注维护数据隐私和安全。法律和伦理要求可能会阻止使用基于云的机器学习解决方案来处理此类任务。在这项工作中，我们将介绍一种将学习到的神经网络转换为CryptoNets的方法，CryptoNets是可以应用于加密数据的神经网络。这允许数据所有者以加密形式将他们的数据发送到托管网络的云服务。加密确保数据保持机密，因为云服务无法访问解密所需的密钥。尽管如此，我们将证明云服务能够将神经网络应用于加密数据以进行加密预测，并且还可以以加密形式返回这些预测。这些加密预测可以发送回持有密钥的秘密所有者，他们可以解密这些预测。因此，云服务不会获得有关原始数据或其做出的预测的任何信息。我们在MNIST光学字符识别任务上展示了CryptoNets。CryptoNets达到99%的准确率，并且可以在单台PC上每小时进行约59000次预测。因此，它们允许高吞吐量、准确和私密的预测。

33rd 国际机器学习会议论文集，纽约，纽约州，美国，2016年。JMLR：第48卷W&CP。版权所有2016年，作者所有。

1. 引言

考虑一家医院，希望使用云服务来预测患者未来30天内再入院的概率，以改善护理质量并降低成本。由于涉及患者信息保密的伦理和法律要求，医院可能被禁止使用此类服务。在这项工作中，我们提出了一种方法，使医院能够在不牺牲患者隐私的情况下使用这项宝贵服务。在提出的协议中，医院加密私人信息，并以加密形式将其发送给预测提供方，在我们的讨论中将其称为云。云能够在加密的数据记录上进行预测计算，并将医院可以解密和阅读的结果返回。加密方案使用公钥进行加密，使用密钥（私钥）进行解密。重要的是要指出，云无法访问密钥，因此它无法解密数据，也无法解密预测。在此过程中，它获得的信息仅仅是它代表医院进行了预测。因此，云可以向医院收取服务费用，但不会了解患者的医疗记录或预测结果。此流程允许进行私密和安全的预测，而无需在数据所有者和服务提供者之间建立信任关系。这可能在医疗、金融、商业以及其他领域有应用。

需要注意的是，这项工作专注于推理阶段。我们假设云端已经有一个模型。在我们的情况下，它将是一个神经网络。

[1]这项工作是在第一作者在微软研究院雷德蒙德期间完成的

was previously trained, for example, using a set of unencrypted data. Training models for these kinds of applications might be a challenge as well due to the same concerns regarding privacy and security. The problem of training such a model is sometimes referred to as *privacy preserving data-mining* (Agrawal & Srikant, 2000). One possible solution to training while preserving privacy lies in the concept of *differential privacy* (Dwork, 2011). Working with a statistical database, differential privacy allows one to control the amount of information leaked from an individual record in a dataset. Therefore, when training, one can use this concept to ensure privacy for any entity whose information is contained in the dataset as well as to create models that do not leak this information about the data they were trained on. However, the notion of differential privacy is not useful in the inference phase since at this stage, we are interested in examining a single record. Other options include working on encrypted data for training as well, in which case either simple classification techniques should be used (Graepel et al., 2013), or other assumptions on the data representation should be made (Aslett et al., 2015a).

The main ingredients of CryptoNets are *homomorphic encryption* and *neural networks*. Homomorphic encryption was originally proposed by Rivest et al. (1978) as a way to encrypt data such that certain operations can be performed on it without decrypting it first. In his seminal paper Gentry (2009) was the first to present a *fully* homomorphic encryption scheme. The term "fully homomorphic" means that the scheme allows arbitrarily many operations to be performed on the encrypted data. Gentry's original scheme was highly inefficient, but since then the work of several researchers has produced significantly more practical schemes. In this work, in particular, we use the homomorphic encryption scheme of Bos et al. (2013), which is very closely related to the schemes in López-Alt et al. (2012); Stehlé & Steinfeld (2011). This scheme is a *leveled* homomorphic encryption scheme, which allows adding and multiplying encrypted messages but requires that one knows in advance the complexity of the arithmetic circuit that is to be applied to the data. In other words, this cryptosystem allows to compute polynomial functions of a fixed maximal degree on the encrypted data. High degree polynomial computation requires the use of large parameters in the scheme, which results in larger encrypted messages and slower computation times. Hence, a primary task in making practical use of this system is to present the desired computation as a low-degree polynomial. We refer the reader to Dowlin et al. (2015); Bos et al. (2013); López-Alt et al. (2012) for details on the encryption scheme, and only give a brief introduction to it in Section 3. We used the *Simple Encrypted Arithmetic Library (SEAL)* for homomorphic encryption[2].

[2]Available at `http://sealcrypto.codeplex.com`

To allow accurate predictions we propose using neural networks, which in recent years have shown great promise for a wide variety of learning tasks. Much of the revival in the interest in neural networks is due to the unprecedented accuracy achieved in tasks such as image classification (Krizhevsky et al., 2012) and speech recognition (Dahl et al., 2012). In Section 2 we present a brief background on neural networks, as well as the necessary adjustments for them to work with homomorphic encryption, thus creating CryptoNets.

One line of criticism against homomorphic encryption is its inefficiency, which is commonly thought to make it impractical for nearly all applications. However, combining together techniques from cryptography, machine learning and software engineering, we show that CryptoNets may be efficient and accurate enough for real world applications. We show that when CryptoNets are applied to the MNIST dataset, an accuracy of 99% can be achieved with a throughput of 58982 predictions per hour on a single PC, and a latency of 250 seconds. Note that a single prediction takes 250 seconds to complete, however, the same process can make 4096 predictions simultaneously with no added cost. Therefore, over an hour, our implementation can make 58982 predictions on average. Hence, CryptoNets are accurate, secure, private, and have a high throughput - an unexpected combination in the realm of homomorphic encryption.

2. Neural Networks

The goal of this work is to demonstrate the application of neural networks over encrypted data. We use the term neural networks to refer to artificial feed-forward neural networks. These networks can be thought of as leveled circuits. Traditionally, these levels are called layers and are visualized as being stacked so that the bottom-most layer is the input layer. Each node of the input layer is fitted with the value of one of the features of the instance at hand. Each of the nodes in the following layers computes a function over the values of the layer beneath it. The values computed at the top-most layer are the outputs of the neural network.

Several common functions can be computed at the nodes. We have listed some of them here:

1. *Weighted-Sum* (convolution layer): Multiply the vector of values at the layer beneath it by a vector of weights and sum the results. The weights are fixed during the inference processes. This function is essentially a dot product of the weight vector and the vector of values of the feeding layer.
2. *Max Pooling*: Compute the maximal value of some of the components of the feeding layer.

CryptoNets

之前可能使用未加密数据集进行过训练。为这类应用训练模型也可能是一个挑战，因为同样存在隐私和安全方面的顾虑。训练此类模型的问题有时被称为隐私保护数据挖掘（Agrawal & Srikant, 2000）。在保护隐私的同时进行训练的一种可能解决方案在于差分隐私（Dwork, 2011）的概念。在与统计数据库工作时，差分隐私允许人们控制数据集中单个记录泄露的信息量。因此，在训练时，可以利用这一概念来确保数据集中任何实体信息的隐私，并创建不会泄露其训练数据信息的模型。然而，差分隐私的概念在推理阶段并不适用，因为在这个阶段，我们感兴趣的是检查单个记录。其他选项包括使用加密数据进行训练，在这种情况下，应该使用简单的分类技术（Graepel 等人，2013），或者对数据表示做出其他假设（Aslett 等人，2015a）。

CryptoNets 的主要成分是同态加密和神经网络。同态加密最初由 Rivest 等人（1978 年）提出，旨在以一种无需先解密即可对其执行某些操作的方式加密数据。在其开创性论文中，Gentry（2009 年）首次提出了一种全同态加密方案。术语“全同态”意味着该方案允许对加密数据执行任意数量的操作。Gentry 的原始方案效率极低，但此后多位研究人员的成果产生了显著更实用的方案。在这项工作中，我们特别使用了 Bos 等人（2013 年）提出的同态加密方案，该方案与 L´opez-Alt 等人（2012 年）；Stehl´e & Steinfeld（2011 年）中的方案非常相似。该方案是一种分层同态加密方案，允许对加密消息进行加法和乘法，但要求事先知道要应用于数据的算术电路的复杂度。换句话说，该密码系统允许在加密数据上计算固定最大次数的多项式函数。高次多项式计算需要在该方案中使用较大的参数，这会导致更大的加密消息和更慢的计算时间。因此，在使用该系统时的一项主要任务是将所需的计算表示为低次多项式。我们建议读者参考 Dowlin 等人（2015 年）；Bos 等人（2013 年）；L´opez-Alt 等人（2012 年）了解有关加密方案的详细信息，并在第 3 节中仅简要介绍它。我们使用了简单加密算术库（SEAL）进行同态加密[2]。

[2]可在http://sealcrypto.codeplex.com获取

为了进行准确预测，我们建议使用神经网络，近年来这些网络在多种学习任务中展现了巨大的潜力。神经网络兴趣的复兴很大程度上归功于在图像分类（Krizhevsky 等人，2012 年）和语音识别（Dahl 等人，2012 年）等任务中取得的前所未有的准确度。在第 2 节中，我们将介绍神经网络的简要背景，以及它们与同态加密协同工作所需的调整，从而创建了 CryptoNets。

对同态加密的一种批评是它的低效性，通常认为这使得它在几乎所有应用中都不切实际。然而，结合密码学、机器学习和软件工程的技术，我们证明 CryptoNets可能足够高效和准确，适用于实际应用。我们证明，当CryptoNets应用于MNIST数据集时，在单台PC上，吞吐量为每小时58982次预测，延迟为250秒即可达到99%的准确率。请注意，单次预测需要250秒完成，但相同过程可以同时进行4096次预测而无需额外成本。因此，在一小时内，我们的实现平均可以进行58982次预测。因此，CryptoNets准确、安全、私密，且具有高吞吐量——在同态加密领域，这是一种意想不到的组合。

2. 神经网络

这项工作的目标是展示神经网络在加密数据上的应用。我们使用术语“神经网络”来指代人工前馈神经网络。这些网络可以被视为分层电路。传统上，这些层级被称为层，并且被可视化地堆叠起来，使得最底层的层是输入层。输入层的每个节点都配备了当前实例的一个特征的值。以下各层的每个节点都计算其下方层值的函数。计算在最顶层得到的值是神经网络的输出。

在节点上可以计算几种常见的函数。我们在这里列出了其中一些：

1. 加权求和（卷积层）：将下方层的值向量与权重向量相乘，并将结果相加。权重在推理过程中是固定的。这个函数本质上就是权重向量和输入层值向量的点积。
2. 最大池化：计算某些区域的最大值喂食层的组件。

3. *Mean Pooling*: Compute the average value of some of the components of the feeding layer.

4. *Sigmoid*: Take the value of one of the nodes in the feeding layer and evaluate the function $z \mapsto 1/(1+\exp(-z))$.

5. *Rectified Linear*: Take the value of one of the nodes in the feeding layer and compute the function $z \mapsto \max(0, z)$.

Since homomorphic encryption supports only additions and multiplications, only polynomial functions can be computed in a straightforward way. Moreover, due to the increased complexity in computing circuits with nested multiplications, it is desired to restrict the computation to low-degree polynomials. The weighted-sum function can be directly implemented since it uses only additions and multiplications. Moreover, the multiplications here are between precomputed weights and the values of the feeding layer. Since the weights are not encrypted, it is possible to use the more efficient plain multiplication operation, as is described in Section 3.2.1. Some networks also add a bias term to the result of the weighted sum. To add this bias term a plain addition can be used since, again, the value of this bias term is known to the cloud.

One thing to note is that the encryption scheme does not support floating-point numbers. Instead, we use fixed precision real numbers by converting them to integers by proper scaling, although there are also other ways to do this (Dowlin et al., 2015). Furthermore, the encryption scheme applies all of its operations modulo some number t, which is why it is important to pay attention to the growth in the size of the numbers appearing throughout the computation, and to make sure that reduction modulo t does not occur. Otherwise the results of the computation might be unexpected. In our experiments, $5-10$ bits of precision on the inputs and weights of the network were sufficient in maintaining the accuracy of the neural network. All the numbers computed were smaller than 2^{80}, which guided us in selecting the parameters for the encryption scheme as seen in Section 3.2.5.

Max pooling cannot be computed directly since the max-function is non-polynomial. However, powers of it can be approximated due to the relation $\max(x_1, \ldots, x_n) = \lim_{d\to\infty}\left(\sum_i x_i^d\right)^{1/d}$. To keep the degree small, d should be kept reasonably small, with the smallest meaningful value $d = 1$ returning a scalar multiple of the mean pooling function. We will use this scaled mean-pool function instead of the max-pool function, as the sum $\sum x_i$ is easy to compute over encrypted data. The reason we use the scaled mean-pool instead of the traditional mean-pool is that we prefer not having to divide by the number of elements, although this could in principle be done (Dowlin et al., 2015). The only effect of not dividing is that the output gets scaled by a factor, which then propagates to the next layers.

The sigmoid and the rectified linear activation functions are non-polynomial functions. The solution of Xie et al. (2014) was to approximate these functions with low-degree polynomials, but we take a different approach here. We try to control the trade-off between having a non-linear transformation, which is needed by the learning algorithm, and the need to keep the degree of the polynomials small, to make the homomorphic encryption parameters feasible. We chose to use the lowest-degree non-linear polynomial function, which is the square function: $\mathrm{sqr}(z) := z^2$. It is interesting to note that Livni et al. (2014) have recently suggested a theoretical analysis of the problem of learning neural networks with polynomial activation functions and devoted much of their study to the square activation function.

In conclusion, to make a network compatible with homomorphic encryption some modifications are needed. Preferably, these modifications should be taken into account while training. The activation functions should be replaced by polynomial activation functions and the max pooling replaced by scaled mean pooling. For the sake of time-efficient evaluation, consecutive layers that use only linear transformations, such as the weighted-sum or mean pooling, can be collapsed.

3. Homomorphic Encryption

Encrypting data is a prominent method for securing and preserving privacy of data. Homomorphic encryption (HE) (Rivest et al., 1978) adds to that the ability to act on the data while it is still encrypted. In mathematics, a *homomorphism* is a *structure-preserving* transformation. For example, consider the map $\Phi : \mathbb{Z} \to \mathbb{Z}_7$ such that $\Phi(z) := z \pmod 7$. This map Φ preserves both the additive and multiplicative structure of the integers in the sense that for every $z_1, z_2 \in \mathbb{Z}$, we have that $\Phi(z_1 + z_2) = \Phi(z_1) \oplus \Phi(z_2)$ and $\Phi(z_1 \cdot z_2) = \Phi(z_1) \otimes \Phi(z_2)$ where $\oplus$ and $\otimes$ are the addition and multiplication operations in $\mathbb{Z}_7$. The map Φ is a ring homomorphism between the rings $\mathbb{Z}$ and $\mathbb{Z}_7$.

In the context of homomorphic encryption, we will be interested in preserving the additive and multiplicative structures of the *rings* of plaintexts and ciphertexts in the encryption and decryption operations. Since the first such encryption scheme was introduced (Gentry, 2009), there have been many advances in this field (see e.g. Naehrig et al. (2011); Gentry et al. (2012a;b); López-Alt et al. (2012)).

Technically speaking, (fully) homomorphic encryption allows for an arbitrary number of addition and multiplication operations to be performed on the encrypted data. For the sake of efficiency, we will instead use a weaker variant

3. MeanPooling：计算某些值的平均值喂食层的组件。

4. Sigmoid：取输入层中一个节点的值并计算函数 $z \mapsto 1/(1+\exp(-z))$。

5. Rectified Linear：取输入层中一个节点的值并计算函数 $z \mapsto \max(0, z)$。

由于同态加密仅支持加法和乘法，因此只能直接计算多项式函数。此外，由于嵌套乘法电路的计算复杂度增加，因此希望将计算限制在低次多项式上。加权和函数可以直接实现，因为它仅使用加法和乘法。此外，这里的乘法是在预计算的权重和输入层的值之间进行的。由于权重未加密，因此可以使用更高效的明文乘法操作，如第3.2.1节所述。一些网络还在加权和的结果中添加一个偏差项。要添加这个偏差项，可以使用明文加法，因为偏差项的值对云端是已知的。

需要注意的一点是，该加密方案不支持浮点数。相反，我们通过适当的缩放将它们转换为整数来使用固定精度的实数，尽管也有其他方法可以这样做（Dowlin 等人，2015 年）。此外，该加密方案对所有操作都应用模某个数 t 的运算，这就是为什么在计算过程中出现的数字大小增长需要特别注意，并确保不会发生模 t 的约减。否则，计算结果可能会出乎意料。在我们的实验中，网络输入和权重上的 $5-10$ 位精度足以保持神经网络的准确性。所有计算出的数字都小于 2^{80}，，这指导我们选择了加密方案的参数，如第 3.2.5 节所示。

最大池化无法直接计算，因为最大值函数是非多项式的。然而，由于 $\max(x_1, \ldots, x_n) = \lim_{d\to\infty}\left(\sum_i x_i^d\right)^{1/d}$ 的关系，其幂次可以近似。为了保持次数较小，d 应保持合理小的值，最小的有意义的值 $d = 1$ 返回均值池化函数的标量倍数。我们将使用这个缩放后的均值池化函数，而不是最大池化函数，因为 $\sum x_i$ 在加密数据上很容易计算。我们使用缩放后的均值池化而不是传统均值池化的原因是，我们倾向于避免除以元素数量，尽管原则上可以这样做（Dowlin 等人，2015 年）。

不除以的唯一影响是输出会被缩放一个因子，然后这个因子会传播到下一层。

Sigmoid 和 ReLU 激活函数是非多项式函数。Xie 等人（2014 年）的解决方案是使用低次多项式来近似这些函数，但我们在这里采取了不同的方法。我们试图控制非线性变换（这是学习算法所必需的）与保持多项式次数较小（以使同态加密参数可行）之间的权衡。我们选择使用最低次的非线性多项式函数，即平方函数：$\mathrm{sqr}(z) := z^2$。值得注意的是，Livni 等人（2014 年）最近提出了对使用多项式激活函数学习神经网络的问题的理论分析，并将他们的大部分研究集中在平方激活函数上。

总之，为了使网络兼容同态加密，需要进行一些修改。最好是在训练时考虑这些修改。激活函数应替换为多项式激活函数，最大池化应替换为缩放均值池化。为了提高评估效率，仅使用线性变换的连续层（如加权求和或均值池化）可以合并。

3. 同态加密

加密数据是保护和保留数据隐私的突出方法。同态加密（HE）（Rivest 等人，1978 年）在此基础上增加了在数据仍然加密时对其进行操作的能力。在数学中，同态是一种保持结构的变换。例如，考虑映射 $\Phi : \mathbb{Z} \to \mathbb{Z}_7$ 使得 $\Phi(z) := z$（模 7）。这个映射 Φ 在某种意义上保留了整数加法和乘法结构，因为对于每个 $z_1\ z_2 \in \mathbb{Z}$，我们有 $\Phi(z_1 + z_2) = \Phi(z_1) \oplus \Phi(z_2)$ 和 $\Phi(z_1 \cdot z_2) = \Phi(z_1) \otimes \Phi(z_2)$，其中 $\oplus$ 和 $\otimes$ 是 $\mathbb{Z}_7$ 中的加法和乘法运算。映射 Φ 是环 $\mathbb{Z}$ 和 $\mathbb{Z}_7$ 之间的环同态。

在同态加密的背景下，我们将关注在加密和解密操作中保持明文环和密文环的加法和乘法结构。自从第一个此类加密方案被引入（Gentry,2009）以来，该领域已经取得了许多进展（例如，参见Naehrig等人（2011）；Gentry等人（2012a;b）；L´opez-Alt等人（2012））。

从技术上讲，(全)同态加密允许对加密数据进行任意次数的加法和乘法运算。为了提高效率，我们将使用一种较弱的变体

of this idea often called leveled homomorphic encryption, where the parameters of the encryption scheme are chosen so that arithmetic circuits of (roughly speaking) a predetermined depth can be evaluated. In our case this amounts to knowing the structure of the neural network, including the activation functions. The particular encryption scheme that we employ is YASHE', described in Bos et al. (2013).

3.1. Description of the method

The encryption scheme of Bos et al. (2013) (also López-Alt et al. (2012)) maps plaintext messages from the ring $R_t^n := \mathbb{Z}_t[x]/(x^n+1)$ to the ring $R_q^n := \mathbb{Z}_q[x]/(x^n+1)$. See the supporting material for a brief introduction to rings and their properties. The encryption scheme chooses random polynomials $f', g \in R_q^n$, and defines $f := tf'+1$. The public key h is defined to be $h := tgf^{-1}$, while f is the secret key. Since not every element in R_q^n is invertible, these steps are iterated until the corresponding f has an inverse and h can be computed.

A message $m \in R_t^n$ is encrypted by computing

$$c := [\lfloor q/t \rfloor m + e + hs]_q$$

where e and s are random noise polynomials in R_q^n, with coefficients of small absolute value. We use the notation $[a]_q$ (resp. $[a]_t$) to denote the reduction of the coefficients of a modulo q (resp. t) to the symmetric interval of length q (resp. t) around 0. Decrypting is done by computing

$$m := \left[\left\lfloor \frac{t}{q} fc \right\rceil\right]_t .$$

Here the product fc is first computed in R_q^n, the coefficients are interpreted as integers, scaled by t/q, and rounded to the nearest integers. Finally they are interpreted modulo t.

Two ciphertexts c_1 and c_2, with underlying messages m_1 and m_2, can be added together in R_q^n to yield the encryption of m_1+m_2. This works because

$$\begin{aligned} c_1 + c_2 = &\lfloor q/t \rfloor (m_1+m_2) + (e_1+e_2) \\ &+ h(s_1+s_2) , \end{aligned} \tag{1}$$

which decrypts to $m_1+m_2 \in R_t^n$.

To multiply two messages we first compute

$$\left\lfloor \frac{t}{q} c_1 c_2 \right\rceil = \lfloor q/t \rfloor (m_1 m_2) + e' + h^2 s_1 s_2 \tag{2}$$

where e' is a noise term that under the right conditions is still small. It is easy to see that the term above decrypts to $m_1 \cdot m_2$, but under the secret key $f^2 \in R_q^n$. Using a process called *relinearization* (see e.g. (Brakerski & Vaikuntanathan, 2011; Bos et al., 2013)), it is possible to modify the result so that it will be decryptable under the original secret key.

3.2. Practical considerations

The first thing to note is that the method described above works as long as the noise terms appearing in the encryptions of m_1 and m_2 are small enough. Otherwise the decryptions might not yield correct answers. The security level of the system depends on the parameters n, q, t, and the amount of noise added. The maximum amount of noise that a ciphertext can have and still be decryptable depends on the parameters q and t.

When ciphertexts are added or multiplied, the noise in the resulting ciphertext is typically larger than in the inputs. Noise growth is particularly strong in multiplication. This essentially means that the parameter q should be selected to be large enough to support the increased noise, which necessitates choosing a larger n for security reasons.

If the computation to be performed is expressed as an arithmetic circuit with addition and multiplication nodes, the main limitation to using the scheme is the number of multiplication gates in the path from the inputs to the outputs. This number we refer to as the *level*. Keeping the level low allows for selecting smaller values for the parameters, which results in faster computation and smaller ciphertexts. Note that the level is not the same as the degree of a polynomial to be evaluated, and instead behaves like the logarithm of the degree.

While keeping the parameters small improves performance for our tasks, we would like to make t large to prevent the coefficients of the plaintext polynomials from reducing modulo t at any point during the computation. To better understand this point, note that the atomic objects used in a neural network are real numbers. The neural network takes as its input a vector of real numbers and, through a series of additions, multiplications, and other real functions, it computes its outputs, which are also real numbers. However, the homomorphic encryption scheme works over the ring $R_t^n := \mathbb{Z}_t[x]/(x^n+1)$. This means that some conversion process between real numbers and elements of R_t^n is needed. We refer to such conversions as encodings (real numbers to R_t^n) and decodings (R_t^n to real numbers). If the coefficients of a polynomial in R_t^n are reduced modulo t after say, an addition, there is usually a problem with decoding it correctly to the sum of the real numbers, and instead the result is likely to be unexpected. This is why we need to keep track of how large the coefficients of the plaintext polynomials grow throughout the entire computation, and choose the parameter t to be larger.

To make the computations faster, it is also important to keep track of what parts of the data need to be secured. A common task that is repeatedly performed in the neural network is computing the weighted some of the inputs from the previous layer. While the data from the previous layer

，这种变体通常称为分层同态加密，其中加密方案的参数被选择为（大致而言）可以评估预定深度的算术电路。在我们的情况下，这意味着需要了解神经网络的架构，包括激活函数。我们采用的特定加密方案是YASHE'，详见Bos等人（2013年）的研究。

3.1. 方法描述

Bos 等人 (2013) 的加密方案（也见 L´opez-Alt 等人 (2012)）将明文消息从环 $R_t^n := \mathbb{Z}_t[x]/(x^n+1)$ 映射到环 $R_q^n := \mathbb{Z}_q[x]/(x^n+1)$。参见支持材料了解环及其性质的简要介绍。该加密方案选择随机多项式 $f'\ g \in R_q^n$、和定义 $f := tf'+1$。公钥 h 定义为 $h := tgf^{-1}$，而 f 是私钥。由于 R_q^n 中的并非每个元素都可逆，因此这些步骤会重复执行，直到相应的 f 具有逆元且 h 可以计算出来。

消息 $m \in R_t^n$ 通过计算加密

$$c := [\lfloor q/t \rfloor m + e + hs]_q$$

其中 e 和 s 是 R_q^n 中的随机噪声多项式，系数的绝对值较小。我们使用符号 $[a]_q$ (resp. $[a]_t$) 表示将 a 的系数模 q (resp. t) 后映射到以 0 为中心、长度为 q (resp. t) 的对称区间。解密通过计算完成

$$m := \left[\left\lfloor \frac{t}{q} fc \right\rceil\right]_t .$$

在此处，产品 fc 首先在 R_q^n 中计算，系数被解释为整数，乘以 t/q，并四舍五入为最接近的整数。最后它们被解释为模 t。

两个密文 c_1 和 c_2，其底层消息为 m_1和 m_2，可以在 R_q^n 中相加，以生成对 m_1+m_2的加密。这之所以可行，是因为

$$\begin{aligned} c_1 + c_2 = &\lfloor q/t \rfloor (m_1+m_2) + (e_1+e_2) \\ &+ h(s_1+s_2) , \end{aligned} \tag{1}$$

它解密为 $m_1+m_2 \in R_t^n$。

要乘以两个消息，我们首先计算

$$\left\lfloor \frac{t}{q} c_1 c_2 \right\rceil = \lfloor q/t \rfloor (m_1 m_2) + e' + h^2 s_1 s_2 \tag{2}$$

在 e' 是一个噪声项，在合适条件下仍然很小。很容易看出上面的项解密为 $m_1 \cdot m_2$，但在密钥 $f^2 \in R_q^n$ 下。使用称为重线性化（例如参见 (Brakerski & Vaikunthanathan, 2011; Bos et al., 2013)）的过程，可以修改结果，使其能够在原始密钥下解密。

3.2. 实际考虑

需要注意的是，上述方法只要在 m_1 和 m_2 的加密中出现的噪声项足够小，就可以正常工作。否则，解密可能无法得到正确的结果。系统的安全级别取决于参数 n、q、t以及添加的噪声量。一个密文最多可以含有多少噪声仍然可以被解密，这取决于参数 q 和 t。

当密文相加或相乘时，结果密文中的噪声通常比输入中的更大。噪声增长在乘法中尤其明显。这基本上意味着参数 q 应该选择得足够大，以支持增加的噪声，这需要出于安全原因选择更大的 n。

如果待执行的计算以包含加法和乘法节点的算术电路形式表示，那么使用该方案的主要限制是输入到输出路径中的乘法门数量。我们将这个数量称为层级。保持层级较低可以选用较小的参数值，从而实现更快的计算和更小的密文。需要注意的是，层级与待评估多项式的次数并不相同，而是表现得像次数的对数。

虽然保持参数较小能提升我们任务的性能，但我们希望使 t 较大，以防止明文多项式的系数在计算过程中任何时刻被模 t 化简。为了更好地理解这一点，请注意神经网络中使用的原子对象是实数。神经网络以实数向量作为输入，并通过一系列加法、乘法和其他实数函数计算其输出，这些输出也是实数。然而，同态加密方案在环 $R_t^n := \mathbb{Z}_t[x]/(x^n+1)$ 上工作。这意味着需要在实数和 R_t^n 元素之间进行某种转换过程。我们将此类转换称为编码（实数到 R_t^n）和解码（R_t^n 到实数）。如果在 R_t^n 中的多项式系数在例如加法操作后被模 t 化简，通常在正确解码为实数和时会出现问题，结果很可能是出乎意料的。这就是为什么我们需要在整个计算过程中跟踪明文多项式系数的增长情况，并选择参数 t 为较大的值。

为了加快计算速度，也需要跟踪哪些数据部分需要被保护。在神经网络中反复执行的一个常见任务是计算前一层的输入加权和。虽然前一层数据是加密的，但权重以明文形式对网络是已知的。因此，在将数据乘以权重时，我们可以使用下面描述的更高效的多乘法形式。

is encrypted, the weights are known to the network in their plain form. Therefore, when multiplying the data by the weights we can use a more efficient form of multiplication, described below.

3.2.1. Plain operations

In applying neural networks a common operation is to add or multiply some value, which is derived from the data with some known constant. The naive way to implement such operations is to first encrypt the constant and then perform the addition or multiplication operation. However, this process is both computationally intensive and adds a large amount of noise if the operation is multiplication. However, this is not necessary. Let $c = \lfloor q/t \rfloor m + e + hs$ be the encrypted message and w the known constant. Addition can be achieved by multiplying w by $\lfloor q/t \rfloor$ and adding that to c, which results in $\lfloor q/t \rfloor (m + w) + e + hs$. This is essentially just encrypting w with no noise and performing normal homomorphic addition.

For multiplication, even the scaling is not needed since $cw = \lfloor q/t \rfloor mw + e' + hs'$. This is very efficient, especially if w is a sparse polynomial. For example, if w is a scalar (as it would be in the scenario below), then this multiplication is computed in linear time in the degree of c, which is $n - 1$.

3.2.2. Encoding

As we already discussed above, there is a mismatch between the atomic constructs in neural networks (real numbers), and the atomic constructs in the homomorphic encryption schemes (polynomials in R_t^n). An encoding scheme should map one to the other in a way that preserves the addition and multiplication operations. Such an encoding scheme can be constructed in several ways. For example, it is possible to convert the real numbers to fixed precision numbers, and then use their binary representation to convert them into a polynomial with the coefficients given by the binary expansion. This polynomial will have the property that when evaluated at 2, it will return the encoded value. Another option is to encode the fixed precision number as a constant polynomial. This encoding is simple, but might seem inefficient in the sense that only one coefficient of the polynomial is being used. In Section 3.2.4 we show how a batch of such instances can be encoded simultaneously to make use of the entire space. One problem with such a *scalar encoding* is that the only coefficient of the message polynomials grows very rapidly when homomorphic operations are performed

3.2.3. Encoding large numbers

As we have already explained, a major challenge for computing in this encryption scheme lies in preventing the coefficients of the plaintext polynomials from overflowing t. This forces us to choose large values for t, which causes the noise to grow more rapidly in the ciphertexts and decreases (with q fixed) the maximum amount of noise tolerated. Therefore, we need to choose a larger q, and subsequently a larger n for security reasons. One way to partially overcome this issue is by using the Chinese Remainder Theorem (CRT) (See the supporting material). The idea is to use multiple primes $t_1, \ldots, t_k$. Given a polynomial $\sum a_i x^i$ we can convert it to k polynomials in such a way that the j-th polynomial is $\sum [a_i \pmod{t_j}] x^i$. Each such polynomial is encrypted and manipulated identically. The CRT guarantees that we will be able to decode back the result, as long as its coefficient does not grow beyond $\prod t_j$. Therefore, this method allows us to encode exponentially large numbers while increasing time and space linearly in the number of primes used.

3.2.4. Parallel Computation

The encryption uses polynomials of a high degree. For example, in our case $n = 4096$, both ciphertext and plaintext polynomials can have degrees up to 4095. If the data is encoded as a scalar, only one out of the 4096 coefficients is being used, while all the operations (additions and multiplications) act on the entire 4096 coefficient polynomials. Therefore, the operations are slow due to the high degree, but the result contains only a single significant coefficient. Another application of the CRT allows us to perform Single Instruction Multiple Data (SIMD) operations at no extra cost Gentry et al. (2012b). Assume that t is selected such that $x^n + 1 \equiv \prod (x - \alpha_i) \pmod{t}$. In this case the CRT can be used to show that $R_t^n \cong \mathbb{Z}_t^{\times n}$. The isomorphism is explicit and easy to compute, which means that we can encode n values into a single polynomial, operate on this polynomial, and decode the n different results.

Note that we use here the CRT in an opposite direction to how we use it when encoding large numbers (Section 3.2.3). When encoding large numbers, we take a single number and break it into multiple small numbers that are being processed in parallel and joined together at the end. On the other hand, here we take multiple scalars and join them together to form a single polynomial. This polynomial is being processed as a single unit and only upon completing the computation is it broken into its components.

3.2.5. Parameter Selection

The main parameters defining the cryptosystem are the plaintext modulus t, the coefficient modulus q and the degree n of the polynomial modulus $(x^n + 1)$. To allow for encoding large enough numbers for the purposes of the network, we used five plaintext moduli and both of the CRT techniques described above. The values used for the q, t

当数据被加密时，权重以明文形式对网络是已知的。因此，在将数据乘以权重时，我们可以使用下面描述的更高效的多乘法形式。

3.2.1. 简单操作

在应用神经网络时，一个常见的操作是对某个值进行加法或乘法，该值是从数据中通过某个已知常数推导出来的。这种操作的朴素实现方式是首先加密常数，然后执行加法或乘法操作。然而，这个过程既计算密集，如果操作是乘法，还会引入大量噪声。但是，这并非必要。设 $c = \lfloor q/t \rfloor m + e + hs$ 为加密消息，w 为已知常数。加法可以通过将 w 乘以 $\lfloor q/t \rfloor$ 并将其加到 c 来实现，结果为 $\lfloor q/t \rfloor (m + w) + e + hs$。这本质上只是对 w 进行无噪声加密，并执行标准的同态加法。

对于乘法，甚至不需要缩放，因为 $cw = \lfloor q/t \rfloor mw + e' + hs'$。这非常高效，特别是如果 w 是稀疏多项式。例如，如果 w 是标量（在以下场景中会是这样），那么这个乘法在 c 的次数内线性时间内计算完成，即 $n - 1$。

3.2.2. 编码

如我们在上文已经讨论的，神经网络中的原子结构（实数）与同态加密方案中的原子结构（ R_t^n中的多项式）之间存在不匹配。编码方案应以一种保留加法和乘法运算的方式将两者映射。这种编码方案可以通过多种方式构建。例如，可以将实数转换为固定精度的数，然后使用它们的二进制表示将其转换为系数由二进制展开式给出的多项式。这个多项式将具有这样的性质：在2处求值时，它会返回编码的值。另一种选择是将固定精度的数编码为常数多项式。这种编码简单，但可能看起来效率不高，因为多项式的系数中只使用了其中一个。在3.2.4节中，我们将展示如何同时编码这样的一批实例以利用整个空间。这种标量编码的一个问题是，消息多项式的唯一系数在执行同态运算时增长非常快

3.2.3. 编码大数

正如我们之前所解释的，这种加密方案中计算的一个主要挑战在于防止共

如我们所解释的，这种加密方案中计算的一个主要挑战在于防止明文多项式的效率溢出 t。这迫使我们选择较大的 t值，这会导致密文中噪声增长更快，并减少（当 q 固定时）可容忍的最大噪声量。因此，我们需要选择更大的 q，随后出于安全原因选择更大的 n 。一种部分克服此问题的方法是使用中国剩余定理（CRT）（参见支持材料）。其思想是使用多个素数 $t_1 \ldots t_k$、。给定一个多项式$\sum a_i x^i$ ，我们可以将其转换为 k 多项式，使得第 j个多项式是 $\sum [a_i$ （模 t_j）$] x^i$。每个这样的多项式都相同地进行加密和处理。CRT保证只要其系数不超过 $\prod t_j$，我们就能解码回结果。因此，这种方法允许我们在增加时间与空间复杂度（相对于所使用的素数数量线性增长）的同时，对指数级大的数进行编码。

3.2.4. 并行计算

该加密使用高次多项式。例如，在我们的案例中 $n = 4096$，密文和明文多项式的次数最高可达 4095。如果数据作为标量编码，则仅使用 4096 个系数中的一个，而所有运算（加法和乘法）都作用于整个 4096 个系数的多项式。因此，由于次数较高，运算速度较慢，但结果仅包含一个显著的系数。CRT 的另一个应用允许我们在不增加额外成本的情况下执行单指令多数据（SIMD）操作 Gentry 等人 (2012b)。假设 t 被选择为满足 $x^n + 1 \equiv \prod (x - \alpha_i) \pmod{t}$。在这种情况下，CRT 可用于证明 $R_t^n \cong \mathbb{Z}_t^{\times n}$。该同构是显式的且易于计算，这意味着我们可以将 n 个值编码到一个多项式中，对该多项式进行运算，并解码出 n 个不同的结果。

请注意，我们在此处使用CRT的方向与我们在编码大数时（第3.2.3节）使用它的方向相反。在编码大数时，我们将一个数分解成多个小数，这些小数并行处理并在最后合并。另一方面，在此处我们取多个标量并将它们合并成一个多项式。这个多项式作为一个整体进行处理，只有在完成计算后才会被分解成其组成部分。

3.2.5. 参数选择

定义该密码系统的主要参数是明文模数 t、系数模数 q 以及多项式模数 $(x^n + 1)$ 的次数 n 。为了在网络中编码足够大的数字，我们使用了五个明文模数以及上述两种CRT技术。用于 q、t 的值对分别是 (t, 40961)、(q , 65537)、(n , 114689)、($(x^n + 1)$, 147457) 和 q_0。t 的值被选择为它们的乘积大于 , 而 足够大以支持所需的精度。同时，它们相对于 的值足够小，以便正确计算所需的网络，即噪声不会增长过大。最后，明文模数被选择为

pairs are $(2^{191}-2589751, 40961)$, $(2^{191}-491527, 65537)$, $(2^{191}-2232751, 114689)$, $(2^{191}-2309565, 147457)$, and $(2^{191}-15131401, 188417)$. The values of t were selected so that their product is greater than 2^{80}, which is large enough to allow the precision needed. At the same time they are small enough, with respect to the values of q to allow for computing the desired network correctly, i.e. so that the noise does not grow too large. Finally, the plaintext moduli are chosen such that

$$x^{4096}+1 = \prod_{i=0}^{4095}\left(x-\alpha_i^{1,\ldots,5}\right) \pmod{t_{1,\ldots,5}}$$

In other words, the polynomial modulus breaks into linear components, which allows for optimal use of the SIMD technique described in Section 3.2.4.

4. Empirical Results

We have tested CryptoNets on the MNIST dataset (LeCun et al., 1998). This dataset consists of 60,000 images of hand written digits. Each image is a 28x28 pixel array, where each pixel is represented by its gray level in the range of 0-255. We used the training part of this dataset, consisting of 50,000 images, to train a network and the remaining 10,000 images for testing. The details of the network used are presented in Table 1. The accuracy of the training network is 99% (it mislabels only 105 out of the $10,000$ test examples).

4.1. Timing analysis

Since the network can accept batches of size 4096 (due to the choice of the degree $n = 4096$ in the encryption parameters) we timed the network on the first 4096 images of the test set to match the batch size. The latency of the network is governed by the time to process a batch while the throughput is also a function of the batch size. Therefore, we separated the report for these two parameters and also reported on the time per instance. These results are presented in Table 2.

Applying the network takes 250 seconds on a PC with a single Intel Xeon E5-1620 CPU running at 3.5GHz, with 16GB of RAM, running the Windows 10 operating system. Since applying the network allows making 4096 predictions simultaneously using the SIMD operations as described in Section 3.2.4, this PC can sustain a throughput of $4096 \times 3600/250 \approx 58982$ predictions per hour. Encrypting the data takes 44.49 seconds and additional 0.138 seconds for every parallel instance to be encoded. Therefore, if 4096 instances are encoded, a throughput of 24193 instances per hour can be encrypted and encoded. Decrypting the data takes 3 seconds and additional 12.4 millisecond to decode predictions for each instance. Therefore, a throughput of 274131 decryptions and decoding per hour is achievable with our setup.

4.2. Description of the Network

The network has two forms: the model that is the direct output of training, and the simplified version which is actually used for making predictions. The trained network has 9 layers, and the simplified version has 5 layers. A visualization of the latter is given in Table 1, though we will describe both of them here.

Here is a description of the network used for training:

1. *Convolution Layer*: The input image is 28×28. The convolution has windows, or kernels, of size 5×5, a stride of $(2, 2)$, and a mapcount of 5. The output of this layer is therefore $5 \times 13 \times 13$.
2. *Square Activation Layer*: This layer squares the value at each input node.
3. *Scaled Mean Pool Layer*: This layer has $1 \times 3 \times 3$ windows, and again outputs a multi-array of dimension $5 \times 13 \times 13$.
4. *Convolution Layer*: This convolution has a kernel size of $1 \times 5 \times 5$, a stride of $(1, 2, 2)$, and a mapcount of 10. The output layer is therefore $50 \times 5 \times 5$.
5. *Scaled Mean Pool Layer*: As with the first mean pool, the kernel size is $1\times3\times3$, and the output is $50\times5\times5$.
6. *Fully Connected Layer*: This layer fully connects the incoming $50 \cdot 5 \cdot 5 = 1250$ nodes to the outgoing 100 nodes, or equivalently, is multiplication by a 100×1250 matrix.
7. *Square Activation Layer*: This layer squares the value at each input node.
8. *Fully Connected Layer*: This layer fully connects the incoming 100 nodes to the outgoing 10 nodes, or equivalently, is multiplication by a 10×100 matrix.
9. *Sigmoid Activation Function*: This layer applies the sigmoid function to each of the 10 incoming values.

The sigmoid activation function is necessary for the training stage in order to get reasonable error terms when running the gradient descent algorithm. However, we don't have a good way of dealing with the sigmoid in the encrypted realm. Luckily, once we have our weights fixed and want to make predictions, we can simply leave it out. This is because the prediction of the neural network is given by the index of the maximum value of its output vector, and since the sigmoid function is monotone increasing, whether or not we apply it will not affect the prediction.

配对为（$2^{191}-2589751$，40961），（$2^{191}-491527$，65537），（$2^{191}-2232751$，114689），（$2^{191}-2309565$，147457），以及$(2^{191}-15131401, 188417)$。$t$ 的值被选择，使得它们的乘积大于 2^{80}，，2^{80}，足够大以允许所需的精度。同时，它们相对于 q 的值足够小，以允许正确计算所需的网络，即使得噪声不会增长过大。最后，明文模数被选择，使得

$$x^{4096}+1 = \prod_{i=0}^{4095}\left(x-\alpha_i^{1,\ldots,5}\right) \pmod{t_{1,\ldots,5}}$$

换句话说，多项式模数会分解成线性分量，这使得能够优化第3.2.4节中描述的SIMD技术。

4. 实验结果

我们在MNIST数据集（LeCun等人，1998年）上测试了CryptoNets。该数据集包含60,000张手写数字图像。每张图像是一个28x28像素的数组，每个像素用0-255范围内的灰度值表示。我们使用了该数据集的训练部分（包含50,000张图像）来训练网络，其余的10,000张图像用于测试。所用网络的详细信息见表1。训练网络的准确率为99%（它在10,000个测试样本中只错误标记了105个）。

4.1. 时间分析

由于网络可以接受大小为4096的批次（这是由于加密参数中选择的度数 $n = 4096$ ），我们在测试集的前4096张图像上进行了计时，以匹配批次大小。网络的延迟由处理一批图像的时间决定，而吞吐量也是批次大小的函数。因此，我们将这两个参数的报告分开，并报告了每个实例的时间。这些结果见表2。

在单颗运行于3.5GHz的Intel Xeon E5-1620 CPU、16GB内存、Windows 10操作系统的PC上应用网络需要250秒。由于应用网络允许使用第3.2.4节中描述的SIMD操作同时进行4096次预测，这台PC可以持续达到每小时 $4096 \times 3600/250 \approx 58982$ 次预测的吞吐量。加密数据需要44.49秒，并且每个并行实例的编码还需要额外0.138秒。因此，如果对4096个实例进行编码，可以每小时加密和编码24193个实例。解密数据需要3秒，并且每个实例的预测解码还需要额外12.4毫秒。因此，

我们的设置可以实现每小时274131次解密和解码的吞吐量。

4.2. 网络描述

该网络有两种形式：一种是训练的直接输出模型，另一种是实际用于进行预测的简化版本。训练后的网络有9层，简化版本有5层。后者的可视化结果见表1，尽管我们在此将描述两者。

以下是用于训练的网络描述：

1. 卷积层：输入图像是 28×28。卷积操作使用大小为 5×5的窗口（或核），步长为(2, 2)，特征图数量为5。因此该层的输出是 $5 \times 13 \times 13$。
2. 平方激活层：该层对每个输入节点的值进行平方处理。
3. 缩放均值池化层：该层具有 $1 \times 3 \times 3$ 窗口，并且再次输出一个维度为 {v1} 的多数组 sion $5 \times 13 \times 13$
4. 卷积层：该卷积层具有核大小为 $1 \times 5 \times 5$、步长为 (1, 2, 2) 以及映射计数为10。因此输出层是 $50 \times 5 \times 5$。
5. ScaledMeanPoolLayer：与第一个meanpool类似核大小是 $1\times3\times3$，输出是 $50\times5\times5$。
6. 全连接层：该层将输入的 $50 \cdot 5 \cdot 5 = 1250$ 个节点完全连接到输出的 100 个节点，或者等价地，是乘以一个 $100 \times\times 1250$ 的矩阵。
7. Square ActivationLayer：该层对值进行平方在每个输入节点上。
8. 全连接层：该层完全连接向输出端10个节点发送100个节点，或等价于，乘以一个 10×100 矩阵。
9. Sigmoid激活函数：该层将Sigmoid函数应用于每个10个输入值。

Sigmoid激活函数对于训练阶段是必要的，以便在运行梯度下降算法时获得合理的误差项。然而，我们在加密领域中还没有处理Sigmoid的好方法。幸运的是，一旦我们的权重固定并想要进行预测，我们可以简单地将其省略。这是因为神经网络的预测由其输出向量的最大值索引给出，并且由于Sigmoid函数是单调递增的，无论我们是否应用它，都不会影响预测。

Table 1. Breakdown of the time it takes to apply CryptoNets to the MNIST network

Layer	Description	Time to compute
Convolution layer	Weighted sums layer with windows of size 5×5, stride size of 2. From each window, 5 different maps are computed and a padding is added to the upper side and left side of each image.	30 seconds
1st square layer	Squares each of the 835 outputs of the convolution layer	81 seconds
Pool layer	Weighted sum layer that generates 100 outputs from the 835 outputs of the 1st square layer	127 seconds
2nd square layer	Squares each of the 100 outputs of the pool layer	10 seconds
Output layer	Weighted sum that generates 10 outputs (corresponding to the 10 digits) from the 100 outputs of the 2nd square layer	1.6 seconds

Table 2. The performance of CryptoNet for MNIST

Stage	Latency	Additional latency per each instance in a batch	Throughput
Encoding+Encryption	44.5 seconds	0.138 seconds	24193 per hour
Network application	250 seconds	0	58982 per hour
Decryption+Decoding	3 seconds	0.012 seconds	274131 per hour

The other change that we make to the network is just for an increase in efficiency. Since layers 3 through 6 are all linear, they can all be viewed as matrix multiplication and composed into a single linear layer corresponding to a matrix of dimension 100 by $5 \cdot 13 \cdot 13 = 865$. Thus, our final network for making predictions is only 5 layers deep.

One obstacle to training networks using the square activation function is that, unlike the rectified linear and sigmoid functions, its derivative is unbounded. This can lead to strange behavior when running the gradient descent algorithm, especially for deeper nets it sometimes blows up or overfits. The overfitting issue can be partially resolved by the addition of convolution layers without activation functions (layers 4 and 5 in our network). This allows us to reduce the number of degrees of freedom in the output polynomial. However, for even deeper nets (10 to 20 layers) something else will be needed to aid in training.

4.3. Message sizes

The images consist of 28×28 pixels. Each pixel is encrypted as 5 polynomials (two values for t are used together with CRT to allow for the large numbers needed). Each coefficient in the polynomial requires 24 bytes and therefore, each image requires $28 \times 28 \times 4096 \times 5 \times 24$ bytes or 367.5 MB. However, the same message can contain 4096 images and therefore, the per image message size is only 91.875 KB. The response of the classifier contains only 10 values (for the 10 possible digits) and therefore the message size is $10 \times 4096 \times 5 \times 24$ which is 4.7 MB or 1.17 KB per image, when 4096 images are encoded together. These numbers are summarized in Table 3.

Table 3. Message sizes of CryptoNet for MNIST

	Message size	Size per instance
Owner $\rightarrow$ Cloud	367.5 MB	91.875 KB
Cloud $\rightarrow$ Owner	4.7 MB	1.17 KB

It is interesting to put the message sizes used in comparison to natural raw representations of these messages. The size of the message depends on the representation used. For example, if each image is represented as an array of size 28x28 and each pixel is represented as a double precision floating point number, then the size of each image is approximately 6 KB, which is 15.3 times smaller than the encrypted version. More concise representation is possible if only a single byte is used to represent each pixel, which will bring the ratio between the encrypted size to the unencrypted size to 120. The sparsity of the data allows compressing the data even further, and indeed the compressed version of this dataset has an average of only 165 bytes per instance. Therefore, the encrypted version is $570\times$ larger than this compressed form. In conclusion, the encrypted data is one to three orders of magnitude larger than the unencrypted data. The exact factor depends on what is considered a natural representation of the data in its raw form.

5. Discussion and Conclusions

The growing interest in *Machine Learning As a Service* (MLAS), where a marketplace of predictors is available on a pay-per-use basis, requires attention to the security and privacy of this model. Not all data types are sensitive, but

表1. 将CryptoNets应用于MNIST网络所需时间的分解

层	描述	计算时间
卷积层	带窗口大小为 5×5 的加权求和层，步长为 2。从每个窗口计算 5 个不同的图，并在每个图像的上方和左侧添加填充。	30 秒
1st 方形层	卷积层的 835 个输出中的每一个都是方形	81 秒
池化层	加权求和层，从第一个平方层的835个输出生成100个输出	127秒
2nd 平方层	池化层的100个输出，每个进行平方运算	10秒
输出层	生成10个输出（对应10个数字）的加权求和来自2的100个输出 nd 平方层	1.6秒

表2. CryptoNet在MNIST上的性能

阶段	延迟	每批每个实例的额外延迟	吞吐量
编码+加密	44.5秒	0.138秒	每小时24193
网络应用	250秒	0	每小时58982
解密+解码	3秒	0.012 秒	每小时 274131 个

我们对网络所做的另一处更改只是为了提高效率。由于第3层到第6层都是线性的，它们都可以看作是矩阵乘法，并组合成一个对应于$100\times 5 \cdot 13 \cdot 13 = 865$维矩阵的单一线性层。因此，我们用于进行预测的最终网络只有5层深。

使用平方激活函数训练网络的一个障碍是，与整流线性函数和Sigmoid函数不同，它的导数是无界的。这会导致在运行梯度下降算法时出现奇怪的行为，特别是对于更深层的网络，有时会爆炸或过拟合。通过添加没有激活函数的卷积层（我们网络中的第4层和第5层），可以部分解决过拟合问题。这允许我们减少输出多项式的自由度。然而，对于更深层的网络（10到20层），还需要其他方法来辅助训练。

4.3. 消息大小

这些图像由 28×28 个像素组成。每个像素被加密为 5 个多项式（t 的两个值与 CRT 结合使用，以允许所需的较大数字）。多项式中的每个系数需要 24 字节，因此，每幅图像需要 $28 \times 28 \times 4096 \times 5 \times 24$ 字节或 367.5 MB。然而，相同的消息可以包含 4096 幅图像，因此，每幅图像的消息大小仅为 91.875 KB。分类器的响应仅包含 10 个值（对应 10 个可能的数字），因此消息大小为 $10 \times 4096 \times 5 \times 24$，当 4096 幅图像一起编码时，每幅图像为 1.17 KB，为 4.7 MB。这些数字汇总在表 3 中。

表3. CryptoNet在MNIST上的消息大小

	消息大小	每个实例的大小
所有者 $\rightarrow$ 云	367.5 MB	91.875 KB
云 $\rightarrow$ 所有者	4.7 MB	1.17 KB

将使用在比较中消息的大小与这些消息的自然原始表示进行比较，这很有趣。消息的大小取决于所使用的表示形式。例如，如果每幅图像表示为 28x28 大小的数组，并且每个像素表示为双精度浮点数，则每幅图像的大小约为 6 KB，这是加密版本的 15.3 倍。如果仅使用一个字节来表示每个像素，则可以更简洁地表示，这将使加密大小与未加密大小的比例达到 120。数据的稀疏性允许进一步压缩数据，事实上，这个数据集的压缩版本每个实例的平均大小仅为 165 字节。因此，加密版本比这种压缩形式大 $570\times$。总之，加密数据比未加密数据大一到三个数量级。确切的因子取决于在原始形式中考虑什么才是数据的自然表示。

5. 讨论与结论

对机器学习即服务（MLAS）的兴趣日益增长，其中预测者市场以按需付费的方式提供，这需要关注该模型的 security 和 privacy。并非所有数据类型都敏感，但

in many applications in medicine, finance, and marketing the relevant data on which predictions are to be made is typically very sensitive.

Different methods can be used to protect the data. For example, the prediction provider and the data owner can encrypt the data while in transit using traditional cryptography. These methods are promising in terms of throughput, latency, and accuracy, but they require some way to establish trust between the cloud and the data owner. The provider also needs to guarantee the safety of the keys, and the safety of the data against attackers while it is stored in the cloud.

Another possible approach would be using secure *Multi-Party Computation* (MPC) techniques (Goldreich, 1998). Most MPC methods establish a communication protocol between the parties involved, such that if the parties follow the protocol they will end with the desired results while protecting the security and privacy of their respective assets (Barni et al., 2006; Orlandi et al., 2007; Piva et al., 2008; Chen & Zhong, 2009). Barni et al. (2006) presented a method of this type, where the data owner encrypts the data and sends it to the cloud. The cloud computes an inner product between the data and the weights of the first layer, and sends the result to the data owner. The data owner decrypts, applies the non-linear transformation, and encrypts the result before sending it back to the cloud. The cloud can apply the second layer and send the output back to the data owner. The process continues until all the layers have been computed. In Orlandi et al. (2007) they also noted that this procedure leaks much of the information of the weights of the network to the data owner, and added a method to obscure the weights. The main difference between these methods and the method we describe in this paper is that in our method the data owner does not have to maintain a constant presence while the neural network is evaluated. For example, the data owner can encrypt the data and store it in the cloud in its encrypted form. The cloud can apply one or several networks to the data while the data owner is offline. Whenever the data owner wishes to read the predictions, it can retrieve the information and decrypt it, allowing the data owner to maintain a much simpler infrastructure. Moreover, since intermediate results are not shared, less information is leaked from the cloud to the data owner.

Graepel et al. (2013) suggested the use of homomorphic encryption for machine learning algorithms. They focused on finding algorithms where the training can be done over encrypted data, and therefore were forced to use learning algorithms in which the training algorithm can be expressed as a low degree polynomial. As a result, most of the algorithms proposed were of the linear discrimination type. Several authors also looked at nearest neighbor classification (Zhan et al., 2005; Qi & Atallah, 2008). However, linear classifiers and nearest neighbor classifiers do not deliver the same level of accuracy that neural networks are capable of delivering.

Aslett et al. (2015a;b) presented ways to train machine learning models over data encrypted with homomorphic encryption. They presented both simple algorithms, such as naive Bayes classifiers, as well as more involved random models such as random forests and some variations of it. Their work differs from our work in several major aspects: The models they propose work well on some tasks, but do not compete well with neural networks on tasks such as recognizing objects in images. They also had to use a unique coding scheme, in which values are compared to threshold before encryption, to allow the learning algorithm to work. CryptoNets imposes fewer requirements on the data owner, and allows the use of neural networks; however, it does not support training on the encrypted data.

Training neural networks over encrypted data is still possible. If all the activation functions are polynomials, and the loss function is polynomial too, back-propagation can be computed using additions and multiplications only. However, there are several challenges in doing so. Computational complexity is a major challenge. Even when trained on plaintext, neural networks are slow to train. Today, much of the effort in the field of machine learning goes towards accelerating this training process by using sophisticated hardware such as GPUs. However, adding homomorphic encryption to the process will make the process at least an order of magnitude slower. It is more likely that the slowdown would be much worse since the level of the computed polynomial is proportional to the number of back-propagation steps made, and therefore using leveled homomorphic encryption does not seem to be feasible. Another challenging aspect in the presence of encryption is the lack of ability of a data scientist to inspect the data and the trained models, to correct mislabeled items, to add features, and to tune the network.

The main contribution of this work is a method that enjoys the accuracy of neural networks with the simplicity of use of homomorphic encryption. By combining techniques from cryptography, machine learning, and engineering, we were able to create a setup in which both accuracy and security are achieved, while maintaining a high level of throughput. This work leaves much room for improvement, however. For example, the throughput and latency can be significantly improved by using GPUs and FPGAs to accelerate the computation. Another direction for further progress would be finding more efficient encoding schemes that allow for smaller parameters, and hence faster homomorphic computation.

在医学、金融和营销的许多应用中，用于预测的相关数据通常非常敏感。

可以使用不同的方法来保护数据。例如，预测提供者和数据所有者可以使用传统密码学在数据传输过程中对数据进行加密。这些方法在吞吐量、延迟和准确性方面很有前景，但它们需要某种方式来建立云和数据所有者之间的信任。提供者还需要保证密钥的安全，以及数据在云中存储时的安全性，以防止攻击者。

另一种可能的方案是使用安全多方计算（MPC）技术（Goldreich，1998）。大多数MPC方法在参与方之间建立通信协议，使得如果参与方遵循该协议，他们最终会得到期望的结果，同时保护各自资产的安全和隐私（Barni等人，2006；Orlandi等人，2007；Piva等人，2008；Chen和Zhong，2009）。Barni等人（2006）提出了一种此类方法，其中数据所有者加密数据并将其发送到云。云计算数据与第一层权重的内积，并将结果发送给数据所有者。数据所有者解密、应用非线性变换，并在将其发送回云之前对结果进行加密。云可以应用第二层并将输出发送回数据所有者。该过程持续进行，直到所有层都计算完毕。在Orlandi等人（2007）中，他们还指出该过程向数据所有者泄露了网络权重的许多信息，并增加了一种方法来混淆权重。这些方法与我们论文中描述的方法的主要区别在于，在我们的方法中，数据所有者不需要在神经网络评估期间保持持续在线。例如，数据所有者可以加密数据并将其以加密形式存储在云中。云可以在数据所有者离线时将一个或多个网络应用于数据。每当数据所有者希望读取预测时，它可以检索信息并解密，使数据所有者能够维护更简单的基础设施。此外，由于中间结果不会被共享，从云泄露给数据所有者的信息更少。

Graepel 等人 (2013) 建议在机器学习算法中使用同态加密。他们专注于寻找可以在加密数据上进行训练的算法，因此不得不使用训练算法可以表示为低次多项式的学习算法。结果，大多数提出的算法都属于线性判别类型。一些作者也研究了最近邻分类 (Zhan 等人，2005；Qi & Atallah，2008)。然而，线性分类器和最近邻分类器无法提供神经网络所能实现的相同级别的精度。

Aslett 等人 (2015a;b) 提出了在用同态加密加密的数据上训练机器学习模型的方法。他们提出了简单的算法，如朴素贝叶斯分类器，以及更复杂的随机模型，如随机森林及其一些变体。他们的工作与我们的工作在几个主要方面有所不同：他们提出的模型在一些任务上表现良好，但在识别图像中的对象等任务上与神经网络相比并不具有竞争力。他们还必须使用一种独特的编码方案，在该方案中，值在加密前与阈值进行比较，以允许学习算法工作。CryptoNets 对数据所有者的要求较少，并允许使用神经网络；然而，它不支持在加密数据上进行训练。

在加密数据上训练神经网络仍然是可能的。如果所有激活函数都是多项式，并且损失函数也是多项式，那么可以使用加法和乘法来计算反向传播。然而，这样做存在几个挑战。计算复杂性是一个主要的挑战。即使在明文上训练，神经网络训练也很慢。如今，机器学习领域的大部分工作都致力于通过使用 GPU 等复杂硬件来加速这一训练过程。然而，将同态加密添加到该过程中会使过程至少慢一个数量级。更有可能的是，减速会更严重，因为计算多项式的级别与反向传播步骤的数量成正比，因此使用分层同态加密似乎并不可行。加密存在时，另一个具有挑战性的方面是数据科学家无法检查数据和训练好的模型，以纠正标记错误的项目，添加特征，以及调整网络。

这项工作的主要贡献是一种结合了神经网络精度和同态加密易用性的方法。通过融合密码学、机器学习和工程学技术，我们成功构建了一个同时实现高精度和高安全性的系统，同时保持了较高的吞吐量。然而，这项工作仍有很大的改进空间。例如，通过使用GPU和FPGA加速计算，可以显著提高吞吐量和降低延迟。另一个进一步发展的方向是寻找更高效的编码方案，以实现更小的参数，从而加速同态计算。

References

Agrawal, Rakesh and Srikant, Ramakrishnan. Privacy-preserving data mining. In *ACM Sigmod Record*, pp. 439–450. ACM, 2000.

Aslett, Louis JM, Esperança, Pedro M, and Holmes, Chris C. Encrypted statistical machine learning: new privacy preserving methods. *arXiv preprint arXiv:1508.06845*, 2015a.

Aslett, Louis JM, Esperança, Pedro M, and Holmes, Chris C. A review of homomorphic encryption and software tools for encrypted statistical machine learning. *arXiv preprint arXiv:1508.06574*, 2015b.

Barni, Mauro, Orlandi, Claudio, and Piva, Alessandro. A privacy-preserving protocol for neural-network-based computation. In *Proceedings of the 8th workshop on Multimedia and security*, pp. 146–151. ACM, 2006.

Bos, Joppe W, Lauter, Kristin, Loftus, Jake, and Naehrig, Michael. Improved security for a ring-based fully homomorphic encryption scheme. In *Cryptography and Coding*, pp. 45–64. Springer, 2013.

Brakerski, Zvika and Vaikuntanathan, Vinod. Efficient fully homomorphic encryption from (standard. In *LWE, FOCS 2011, IEEE 52nd Annual Symposium on Foundations of Computer Science, IEEE*. Citeseer, 2011.

Chen, Tingting and Zhong, Sheng. Privacy-preserving backpropagation neural network learning. *Neural Networks, IEEE Transactions on*, 20(10):1554–1564, 2009.

Dahl, George E, Yu, Dong, Deng, Li, and Acero, Alex. Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition. *Audio, Speech, and Language Processing, IEEE Transactions on*, 20(1):30–42, 2012.

Dowlin, Nathan, Gilad-Bachrach, Ran, Laine, Kim, Lauter, Kristin, Naehrig, Michael, and Wernsing, John. Manual for using homomorphic encryption for bioinformatics. Technical report, Microsoft Research, 2015. http://research.microsoft.com/apps/pubs/default.aspx?id=258435.

Dwork, Cynthia. Differential privacy. In *Encyclopedia of Cryptography and Security*, pp. 338–340. Springer, 2011.

Gentry, Craig. Fully homomorphic encryption using ideal lattices. In *STOC*, volume 9, pp. 169–178, 2009.

Gentry, Craig, Halevi, Shai, and Smart, Nigel P. Fully homomorphic encryption with polylog overhead. In *Advances in Cryptology–EUROCRYPT 2012*, pp. 465–482. Springer, 2012a.

Gentry, Craig, Halevi, Shai, and Smart, Nigel P. Homomorphic evaluation of the aes circuit. In *Advances in Cryptology–CRYPTO 2012*, pp. 850–867. Springer, 2012b.

Goldreich, Oded. Secure multi-party computation. *Manuscript. Preliminary version*, 1998.

Graepel, Thore, Lauter, Kristin, and Naehrig, Michael. Ml confidential: Machine learning on encrypted data. In *Information Security and Cryptology–ICISC 2012*, pp. 1–21. Springer, 2013.

Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pp. 1097–1105, 2012.

LeCun, Yan, Cortes, Corinna, and Burges, Christopher J.C. The mnist database of handwritten digits. http://yann.lecun.com/exdb/mnist/, 1998.

Livni, Roi, Shalev-Shwartz, Shai, and Shamir, Ohad. On the computational efficiency of training neural networks. In *Advances in Neural Information Processing Systems*, pp. 855–863, 2014.

López-Alt, Adriana, Tromer, Eran, and Vaikuntanathan, Vinod. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pp. 1219–1234. ACM, 2012.

Naehrig, Michael, Lauter, Kristin, and Vaikuntanathan, Vinod. Can homomorphic encryption be practical? In *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, pp. 113–124. ACM, 2011.

Orlandi, Claudio, Piva, Alessandro, and Barni, Mauro. Oblivious neural network computing via homomorphic encryption. *EURASIP Journal on Information Security*, 2007:18, 2007.

Piva, Alessandro, Orlandi, Claudio, Caini, M, Bianchi, Tiziano, and Barni, Mauro. Enhancing privacy in remote data classification. In *Proceedings of The Ifip Tc 11 23rd International Information Security Conference*, pp. 33–46. Springer, 2008.

Qi, Yinian and Atallah, Mikhail J. Efficient privacy-preserving k-nearest neighbor search. In *Distributed Computing Systems, 2008. ICDCS'08. The 28th International Conference on*, pp. 311–319. IEEE, 2008.

Rivest, Ronald L, Adleman, Len, and Dertouzos, Michael L. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.

CryptoNets

参考文献

Agrawal, Rakesh and Srikant, Ramakrishnan. Privacy-preserving data mining. In *ACM Sigmod Record*, pp. 439–450. ACM, 2000.

Aslett, Louis JM, Esperança, Pedro M, and Holmes, Chris C. Encrypted statistical machine learning: new privacy preserving methods. *arXiv preprint arXiv:1508.06845*, 2015a.

Aslett, Louis JM, Esperança, Pedro M, and Holmes, Chris C. A review of homomorphic encryption and software tools for encrypted statistical machine learning. *arXiv preprint arXiv:1508.06574*, 2015b.

Barni, Mauro, Orlandi, Claudio, and Piva, Alessandro. A privacy-preserving protocol for neural-network-based computation. In *Proceedings of the 8th workshop on Multimedia and security*, pp. 146–151. ACM, 2006.

Bos, Joppe W, Lauter, Kristin, Loftus, Jake, and Naehrig, Michael. Improved security for a ring-based fully homomorphic encryption scheme. In *Cryptography and Coding*, pp. 45–64. Springer, 2013.

Brakerski, Zvika and Vaikuntanathan, Vinod. Efficient fully homomorphic encryption from (standard. In *LWE, FOCS 2011, IEEE 52nd Annual Symposium on Foundations of Computer Science, IEEE*. Citeseer, 2011.

Chen, Tingting and Zhong, Sheng. Privacy-preserving backpropagation neural network learning. *Neural Networks, IEEE Transactions on*, 20(10):1554–1564, 2009.

Dahl, George E, Yu, Dong, Deng, Li, and Acero, Alex. Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition. *Audio, Speech, and Language Processing, IEEE Transactions on*, 20(1):30–42, 2012.

Dowlin, Nathan, Gilad-Bachrach, Ran, Laine, Kim, Lauter, Kristin, Naehrig, Michael, and Wernsing, John. Manual for using homomorphic encryption for bioinformatics. Technical report, Microsoft Research, 2015. http://research.microsoft.com/apps/pubs/default.aspx?id=258435.

Dwork, Cynthia. Differential privacy. In *Encyclopedia of Cryptography and Security*, pp. 338–340. Springer, 2011.

Gentry, Craig. Fully homomorphic encryption using ideal lattices. In *STOC*, volume 9, pp. 169–178, 2009.

Gentry, Craig, Halevi, Shai, and Smart, Nigel P. Fully homomorphic encryption with polylog overhead. In *Advances in Cryptology–EUROCRYPT 2012*, pp. 465–482. Springer, 2012a.

Gentry, Craig, Halevi, Shai, and Smart, Nigel P. Homomorphic evaluation of the aes circuit. In *Advances in Cryptology–CRYPTO 2012*, pp. 850–867. Springer, 2012b.

Goldreich, Oded. Secure multi-party computation. *Manuscript. Preliminary version*, 1998.

Graepel, Thore, Lauter, Kristin, and Naehrig, Michael. Ml confidential: Machine learning on encrypted data. In *Information Security and Cryptology–ICISC 2012*, pp. 1–21. Springer, 2013.

Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pp. 1097–1105, 2012.

LeCun, Yan, Cortes, Corinna, and Burges, Christopher J.C. The mnist database of handwritten digits. http://yann.lecun.com/exdb/mnist/, 1998.

Livni, Roi, Shalev-Shwartz, Shai, and Shamir, Ohad. On the computational efficiency of training neural networks. In *Advances in Neural Information Processing Systems*, pp. 855–863, 2014.

López-Alt, Adriana, Tromer, Eran, and Vaikuntanathan, Vinod. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pp. 1219–1234. ACM, 2012.

Naehrig, Michael, Lauter, Kristin, and Vaikuntanathan, Vinod. Can homomorphic encryption be practical? In *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, pp. 113–124. ACM, 2011.

Orlandi, Claudio, Piva, Alessandro, and Barni, Mauro. Oblivious neural network computing via homomorphic encryption. *EURASIP Journal on Information Security*, 2007:18, 2007.

Piva, Alessandro, Orlandi, Claudio, Caini, M, Bianchi, Tiziano, and Barni, Mauro. Enhancing privacy in remote data classification. In *Proceedings of The Ifip Tc 11 23rd International Information Security Conference*, pp. 33–46. Springer, 2008.

Qi, Yinian and Atallah, Mikhail J. Efficient privacy-preserving k-nearest neighbor search. In *Distributed Computing Systems, 2008. ICDCS'08. The 28th International Conference on*, pp. 311–319. IEEE, 2008.

Rivest, Ronald L, Adleman, Len, and Dertouzos, Michael L. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.

Stehlé, Damien and Steinfeld, Ron. Making ntru as secure as worst-case problems over ideal lattices. In *Advances in Cryptology–EUROCRYPT 2011*, pp. 27–47. Springer, 2011.

Xie, Pengtao, Bilenko, Misha, Finley, Tom, Gilad-Bachrach, Ran, Lauter, Kristin, and Naehrig, Michael. Crypto-nets: Neural networks over encrypted data. *arXiv preprint arXiv:1412.6181*, 2014.

Zhan, Justin Zhijun, Chang, LiWu, and Matwin, Stan. Privacy preserving k-nearest neighbor classification. *IJ Network Security*, 1(1):46–51, 2005.

CryptoNets

Stehlé, Damien and Steinfeld, Ron. Making ntru as secure as worst-case problems over ideal lattices. In *Advances in Cryptology–EUROCRYPT 2011*, pp. 27–47. Springer, 2011.

Xie, Pengtao, Bilenko, Misha, Finley, Tom, Gilad-Bachrach, Ran, Lauter, Kristin, and Naehrig, Michael. Crypto-nets: Neural networks over encrypted data. *arXiv preprint arXiv:1412.6181*, 2014.

Zhan, Justin Zhijun, Chang, LiWu, and Matwin, Stan. Privacy preserving k-nearest neighbor classification. *IJ Network Security*, 1(1):46–51, 2005.